

Аналитический отчёт

EDA

В ноутбуке используется датасет на 1001 объект и 214 признаков, после удаления служебного столбца Unnamed, остаётся 213 рабочих признака, включая три таргет переменные. Так как датасет медицинский заполняю пропуски 0.

Представлены данные о химических соединениях с указанием их эффективности против вируса гриппа. Параметры, характеризующие эффективность, обозначаются как IC50, CC50 и SI, это и есть три целевые переменные. SI рассчитывается на основе параметров IC50 и CC50.

Анализ распределение был получен с помощью методов гистограмм и boxplot. Из графиков видно что целевые переменные IC50, CC50 и SI распределены с большим правым смещением. И видно что есть выбросы, по IQR-правилу доля выбросов составляет 14.7% для IC50, 3.9% для CC50 и 12.5% для SI. Выбросы не удаляли, так как это медицинский датасет. Были проанализированы признаки, это набор молекулярных дескрипторов и фрагментных признаков для молекул. Сами объекты это молекула. В признаках описывается есть ли фрагмент в молекуле и сколько раз встречается, также еще описывают обычные физико-химические дескрипторы числового типа. IC50 служит для определения минимальной ингибирующей концентрации, необходимой для подавления 50% патогена, а CC50 используем как цитотоксическую концентрацию экстрактов, вызывающую гибель 50% жизнеспособных клеток хозяина. SI это индексное значение, дающее представление о селективности. SI рассчитывается на основе параметров IC50 и CC50.

Видно что есть большая размерность. С помощью корреляции Пирсона, мы построили матрицу корреляции Пирсона, из heatmap видно что есть довольно большое количество мультиколлинеарных признаков, которые негативно влияют на построение базовых линейных моделей. Получилось что признаков для исключения IC50 – 49 признаков, признаков для исключения CC50 - 48 признаков, признаков для исключения SI - 46 признаков.

Для каждого таргета исключали свои признаки, для уменьшения вклада в объяснённую дисперсию. Добавлен класс SelectiveSkewTransformer это трансформер который внедряем в pipeline, он нужен чтобы избежать утечки данных в процессе преобразования, который берет список колонок с правой и с левой асимметрией и исправляет ее через PowerTransformer.

Для каждой целевой переменной использовал базовую модель линейная регрессия с L2 регуляцией. Модель для CC50 наиболее производительна train R2 = 0.5145, test R2 = 0.4739 переобучение всего 0.0406, что укладывается в погрешность. Для IC50 производительность модели заметно слабее train R2 = 0.4692, test R2 = 0.3147, переобучение 0.1546, модель заметно лучше объясняет тренировочные данные, на новых теряет качество, по r2 видно что предсказывает не многим лучше среднего. Для SI результат слабый train R2 = 0.1270, test R2 = 0.0366. Я думаю модель плохо описывает данные поскольку SI не является независимой целевой переменной, а рассчитывается как отношение CC50 / IC50. Для задач регрессии и классификации, для каждой целевой переменной сохраняем файл с обработанными данными.

Рекомендации по продолжению исследования: можно пересмотреть заполнение пропусков, заменить на медианные значения или отдельная стратегия по группам признаков. Проверить более сильные модели такие как ансамбли.

Регрессия для IC50

В регрессии для IC50 используется датасет из 161 признака и 1 целевой переменной target. Данные разбиты на train/test в пропорции 90/10. Сам датасет уже подготовлен в EDA и сохранен в csv.

За базовую модель я использую линейную регрессию с L2 регуляризацией, так как довольно много нелинейных связей в датасете. Использовал GridSearchCV, кросс-валидация = 5, лучшие параметры alpha=100. И получаю R2 на обучающей выборке = 0.46, R2 на тестовой выборке = 0.31. Производительность базовой модели низкая, не

подходит для наших задач. Есть переобучение 0.15. Угадывает на новых данных чуть хуже чем среднее. Для сравнения я взял еще линейную регрессию с L1 регуляризацией. Использовал GridSearchCV, кросс-валидация = 5, лучшие параметры $\alpha = 1$. И получаю немного лучше результаты R^2 на обучающей выборке = 0.50, R^2 на тестовой выборке = 0.34. Производительность модели тоже низкая. Есть переобучение 0.16.

Угадывает на новых данных чуть хуже чем среднее.

Еще для сравнения взял 2 модели ансамблей RandomForest и XGBoost.

Для RandomForest использовал GridSearchCV, кросс-валидация = 5, лучшие параметры: максимальная глубина дерева = 10, число признаков при разбиении = $\sqrt{}$, минимальное число объектов в листе = 2, минимальное число объектов, необходимое для разбиения узла = 2, число деревьев = 300. И получаю R^2 на обучающей выборке = 0.74, R^2 на тестовой выборке = 0.45. Со стандартными параметрами переобучение еще больше, удалось добиться нормальной производительности модели, хоть есть и переобучение 0.28, предполагаю что если бы было больше данных, можно добиться хороших показателей и на тесте.

Визуализировал на графике barplot важность признаков, какие признаки дают больший вклад в объясненную дисперсию, а какие меньше. Но так как это медицинские данные нельзя просто признаки удалять.

Для XGBoost использовал RandomizedSearchCV, так как GridSearchCV подбирал очень долго, лучшие параметры: доля объектов для каждого дерева = 0.8, число деревьев = 300, максимальная глубина дерева = 3, скорость обучения = 0.05, коэффициент L2-регуляризации = 5, доля признаков для каждого дерева = 1, коэффициент L1-регуляризации = 1. И получаю R^2 на обучающей выборке = 0.79, R^2 на тестовой выборке = 0.50. XGBoost себя лучше чем линейные модели, хоть есть и переобучение 0.24, предполагаю что если бы было больше данных, можно добиться хороших показателей и на тесте.

Дальше визуализирую на графике сравнение переобучения которое есть у всех моделей, связано с малым количеством объектов, у линейных моделей переобучение меньше, чем у ансамблей. На следующем графике визуализирую сравнение ошибки MSE, которая высокая на всех моделях так как есть большие выбросы, меньше всего MSE у XGBoost, больше у базовой модели. На следующем графике визуализирую сравнение R^2 . Лучше всего на XGBoost, хуже на базовой модели. Так как шумные данные и ненормально распределенные, линейные модели показали себя хуже, предсказывают на новых данных лучше чем среднее. Ансамбли лучше, лучше всего производительность показала модель XGBRegressor, у нее хороший R^2 и меньше переобучение. Имея больше данных производительность моделей можно увеличить.

Рекомендации по продолжению исследования: можно попробовать log преобразование целевой переменной. Провести аналогичные исследования с большим количеством примеров.

Регрессия для CC50

В регрессии для CC50 используется датасет из 161 признака и 1 целевой переменной target. Данные разбиты на train/test в пропорции 90/10. Сам датасет уже подготовлен в EDA и сохранен в csv.

За базовую модель я использую линейную регрессию с L2 регуляризацией, так как довольно много нелинейных связей в датасете. Использовал GridSearchCV, кросс-валидация = 5, лучшие параметры $\alpha = 10$. И получаю R^2 на обучающей выборке = 0.56, R^2 на тестовой выборке = 0.54. Производительность базовой модели средняя. Переобучения нет 0.01, объясняет лучше среднего. Это средний результат для задачи. С учетом низкого переобучения то модель обладает обобщающей способностью и является стабильной.

Для сравнения я взял еще линейную регрессию с L1 регуляризацией. Использовал GridSearchCV, кросс-валидация = 5, лучшие параметры $\alpha = 1$. И получаю немного лучше результаты R^2 на обучающей выборке = 0.56, R^2 на тестовой выборке = 0.53. Производительность модели тоже средняя. Нет переобучение 0.03. Угадывает на новых данных чуть лучше чем среднее. Это средний результат для задачи. С учетом низкого переобучения то модель обладает обобщающей способностью и является стабильной.

Еще для сравнения взял 2 модели ансамблей RandomForest и XGBoost.

Для RandomForest использовал GridSearchCV, кросс-валидация = 5, лучшие параметры: максимальная глубина дерева = 10, число признаков при разбиении = 0.5, минимальное число объектов в листе = 1, минимальное число объектов, необходимое для разбиения узла = 5, число деревьев = 100. И получаю R2 на обучающей выборке = 0.82, R2 на тестовой выборке = 0.65. Со стандартными параметрами переобучение больше, удалось добиться хорошей производительности модели, хоть есть и небольшое переобучение 0.16, предполагаю что если бы было больше данных, можно добиться лучших показателей и на тесте.

Визуализировал на графике barplot важность признаков, какие признаки дают больший вклад в объясненную дисперсию, а какие меньше. Но так как это медицинские данные нельзя просто признаки удалять.

Для XGBoost использовал RandomizedSearchCV, кросс-валидация = 5, так как GridSearchCV подбирал очень долго, лучшие параметры: доля объектов для каждого дерева = 0.8, число деревьев = 300, максимальная глубина дерева = 3, скорость обучения = 0.1, коэффициент L2-регуляризации = 5, доля признаков для каждого дерева = 0.7, коэффициент L1-регуляризации = 0. И получаю R2 на обучающей выборке = 0.83, R2 на тестовой выборке = 0.65. XGBoost показал хорошую производительность, заметно лучше чем линейные модели, хоть есть и небольшое переобучение 0.17.

Дальше визуализирую на графике сравнение переобучения которое есть небольшое у ансамблей, у линейных моделей переобучение нет.

На следующем графике визуализирую сравнение ошибки MSE, которая высокая на всех моделях так как есть большие выбросы, меньше всего MSE у XGBoost, больше у базовой модели.

На следующем графике визуализирую сравнение R2. Лучше всего на XGBoost и RandomForest, хуже на базовой модели. Так как шумные данные и ненормально распределенные, линейные модели показали себя хуже, предсказывают на новых данных лучше чем среднее. Ансамбли лучше, лучше всего производительность показала модель XGBoost, у нее хороший R2, небольшое переобучение переобучение и хорошая производительность, для такого датасета.

Рекомендации по продолжению исследования: тоже можно попробовать log преобразование целевой переменной. И также Провести аналогичные исследования с большим количеством примеров. Хотя с производительностью все хорошо.

Регрессия для SI

В регрессии для SI используется датасет из 164 признака и 1 целевой переменной target. Данные разбиты на train/test в пропорции 90/10. Сам датасет уже подготовлен в EDA и сохранен в csv.

За базовую модель я использую линейную регрессию с L2 регуляризацией, так как довольно много нелинейных связей в датасете. Использовал GridSearchCV, кросс-валидация = 5, лучшие параметры alpha = 100. И получаю R2 на обучающей выборке = 0.11, R2 на тестовой выборке = 0.11. Производительность базовой модели средняя.

Переобучения нет, объясняет почти хуже чем среднее. Это низкий результат для задачи.

Для сравнения я взял еще линейную регрессию с L1 регуляризацией. Использовал GridSearchCV, кросс-валидация = 5, лучшие параметры alpha = 100. И получаю еще хуже результаты R2 на обучающей выборке = 0.00, R2 на тестовой выборке = - 0.003.

Производительность низкая, объясняет хуже чем среднее. Нет переобучение.

Еще для сравнения взял 2 модели ансамблей RandomForest и XGBoost.

Для RandomForest использовал GridSearchCV, кросс-валидация = 5, лучшие параметры: максимальная глубина дерева = 3, число признаков при разбиении = log2, минимальное число объектов в листе = 4, минимальное число объектов, необходимое для разбиения узла = 2, число деревьев = 100. И получаю R2 на обучающей выборке = 0.12, R2 на тестовой выборке = 0.07. У модели низкая производительность модели, объясняет почти хуже чем среднее. Переобучения в рамках погрешности 0.05.

Визуализировал на графике barplot важность признаков, какие признаки дают больший вклад в объясненную дисперсию, а какие меньше. Но так как это медицинские данные нельзя просто признаки удалять.

Для XGBoost использовал RandomizedSearchCV, так как GridSearchCV подбирал очень долго, кросс-валидация = 5, лучшие параметры: доля объектов для каждого дерева = 1, число деревьев = 100, максимальная глубина дерева = 7, скорость обучения = 0.05, коэффициент L2-регуляризации = 10, доля признаков для каждого дерева = 0.8, коэффициент L1-регуляризации = 0.1. И получаю R2 на обучающей выборке = 0.16, R2 на тестовой выборке = 0.11. Низкая производительность модели, объясняет почти хуже чем среднее. Переобучение в рамках погрешности 0.05.

Дальше визуализирую на графике сравнение переобучения которое есть небольшое у ансамблей, у линейных моделей переобучение нет.

На следующем графике визуализирую сравнение ошибки MSE, которая высокая на всех моделей так как есть большие выбросы, меньше всего MSE у XGBoost и у базовой модели, больше у линейной регрессии с L1-регуляризацией.

На следующем графике визуализирую сравнение R2. Хуже всего показала модель линейная регрессия с L1-регуляризацией, лучше всего линейная регрессия с L2-регуляризацией, все модели показали не очень хорошие результаты.

Приемлемой производительности добиться не получилось ни на одной модели, текущий набор признаков объясняет SI слабо.

Рекомендации по продолжению исследования: можно попробовать log преобразование целевой переменной. Провести аналогичные исследования с большим количеством примеров.

Классификация: превышает ли значение IC50 медианное значение выборки

В классификации используется данные которые разбиты на train/test в пропорции 80/20. Сам датасет уже подготовлен в EDA и сохранен в csv. Этот датасет состоит примерно из 1001 объект и 165 столбцов, из которых целевой переменной - median_threshold, а также из признаков исключаются IC50, mM, CC50, mM и SI перед обучением моделей.

Визуализирую с помощью histplot распределение классов, видно что распределение по признаку median_threshold сбалансировано, класс 1 - значит превышает значение IC50 медианное значение выборки, класс 0 - не превышает.

За базовую модель беру логистическую регрессию. Использую StandardScaler. У модели есть переобучение, модель лучше находит класс который превышает медианное значение recall = 0.76, чем класс который не превышает, так как recall выше, хуже распознает класс который меньше чем медианное значение recall = 0.6, часть объектов ошибочно относятся к классу который превышает медианное значение. Ошибается модель больше для класс который превышает медианное значение precision = 0.66, против класса который меньше чем медианное значение precision = 0.72.

Производительность модели не высокая f1 = 0.68.

Еще для сравнения взял 2 модели ансамблей RandomForest и XGBoost.

Для XGBoost использовал RandomizedSearchCV, так как GridSearchCV подбирал очень долго, кросс-валидация = 3, лучшие параметры: доля объектов для каждого дерева = 0.8, число деревьев = 200, максимальная глубина дерева = 10, скорость обучения = 0.01, коэффициент L2-регуляризации = 10, доля признаков для каждого дерева = 0.7, коэффициент L1-регуляризации = 1. для XGBoost есть переобучение, модель лучше находит класс который превышает медианное значение recall = 0.8, чем класс который не превышает, так как recall выше, хуже распознает класс который меньше чем медианное значение recall = 0.65, часть объектов ошибочно относятся к классу который превышает медианное значение. Ошибается модель больше для класс который превышает медианное значение precision = 0.70, против класса который меньше чем медианное значение precision = 0.77. Производительность модели нормальная f1 = 0.73.

Для RandomForest использовал RandomizedSearchCV, кросс-валидация = 3, лучшие параметры: число деревьев = 100, минимальное число объектов, необходимое для разбиения узла = 2, минимальное число объектов в листе = 1, число признаков при разбиении = log2, максимальная глубина дерева = 7. Для RandomForest есть переобучение, модель лучше находит класс который превышает медианное значение recall = 0.78, чем класс который не превышает, так как recall выше, хуже распознает класс который меньше чем медианное значение recall = 0.65, часть объектов ошибочно

относиться к классу который превышает медианное значение. Ошибается модель больше для класс который превышает медианное значение $\text{precision} = 0.69$, против класса который меньше чем медианное значение $\text{precision} = 0.75$. Производительность модели нормальная $f1 = 0.72$.

Дальше визуализирую на графике сравнение метрик всех моделей на обучение, где лучше показывает себя RandomForest, остальные модели тоже показывают нормальную производительность.

На следующем графике визуализирую сравнение метрик всех моделей на тестовой выборки где лучше показал себя XGBoost, но остальные не сильно отстают.

На следующем графике визуализирую сравнения моделей по ROC-AUC, где площадь под кривой лучше у ансамблей, хуже у логистической регрессии, но все три модели лучше случайного угадывания, потому что их ROC-кривые проходят выше диагонали случайного классификатора. 0.79 — это уже нормальный рабочий уровень, но не очень высокий, мало данных, при больших данных можно улучшить.

На следующем графике визуализирую сравнения моделей показатель переобучения $f1$, где у всех трех моделей есть переобучение по $f1$, у логистической регрессии оно минимально, хуже всего у RandomForest.

На следующем графике визуализирую сравнения моделей показатель переобучения accuracy, где у всех трех моделей есть переобучение по accuracy, у логистической регрессии оно минимально, хуже всего у RandomForest.

Ансамбли показали лучше чем логистическая регрессия, у логистической регрессии меньше всего переобучение, модель XGBoost показывает лучшую производительность на тесте и при этом переобучение ниже чем RandomForest.

Рекомендации по продолжению исследования: Провести аналогичные исследования с большим количеством примеров. Можно изменить соотношение train/test.

Классификация: превышает ли значение CC50 медианное значение выборки

В классификации используется данные которые разбиты на train/test в пропорции 80/20. Сам датасет уже подготовлен в EDA и сохранен в csv. Этот датасет состоит примерно из 1001 объект и 166 столбцов, из которых целевой переменной - median_threshold, а также из признаков исключаются IC50, mM, CC50, mM и SI перед обучением моделей.

Визуализирую с помощью histplot распределение классов, видно что распределение по признаку median_threshold сбалансировано, класс 1 - значит превышает значение IC50 медианное значение выборки, класс 0 - не превышает.

За базовую модель беру логистическую регрессию. Использую StandardScaler. У модели есть небольшое переобучение, модель лучше находит класс который превышает медианное значение $\text{recall} = 0.8$, чем который не превышает, так как recall выше, хуже распознает класс который меньше чем медианное значение $\text{recall} = 0.66$, часть объектов ошибочно относятся к классу который превышает медианное значение. Ошибается модель больше для класс который превышает медианное значение $\text{precision} = 0.70$, против класса который меньше чем медианное значение $\text{precision} = 0.77$.

Производительность модели нормальная $f1 = 0.73$.

Еще для сравнения взял 2 модели ансамблей RandomForest и XGBoost.

Для XGBoost использовал RandomizedSearchCV, так как GridSearchCV подбирал очень долго, кросс-валидация = 3, лучшие параметры: доля объектов для каждого дерева = 1, число деревьев = 300, максимальная глубина дерева = 3, скорость обучения = 0.01, коэффициент L2-регуляризации = 10, доля признаков для каждого дерева = 0.7, коэффициент L1-регуляризации = 0. для XGBoost есть переобучение, модель лучше находит класс который превышает медианное значение $\text{recall} = 0.77$, чем который не превышает, так как recall выше, хуже распознает класс который меньше чем медианное значение $\text{recall} = 0.61$, часть объектов ошибочно относятся к классу который превышает медианное значение. Ошибается модель больше для класс который превышает медианное значение $\text{precision} = 0.66$, против класса который меньше чем медианное значение $\text{precision} = 0.73$. Производительность модели нормальная $f1 = 0.69$.

Для RandomForest использовал RandomizedSearchCV, кросс-валидация = 3, лучшие параметры: число деревьев = 100, минимальное число объектов, необходимое для разбиения узла = 10, минимальное число объектов в листе = 4, число признаков при разбиении = $\log_2$, максимальная глубина дерева = 7. Для RandomForest есть переобучение, модель лучше находит класс который превышает медианное значение $\text{recall} = 0.81$, чем класс который не превышает, так как recall выше, хуже распознает класс который меньше чем медианное значение $\text{recall} = 0.61$, часть объектов ошибочно относятся к классу который превышает медианное значение. Ошибается модель больше для класс который превышает медианное значение $\text{precision} = 0.68$, против класса который меньше чем медианное значение $\text{precision} = 0.77$. Производительность модели нормальная $f1 = 0.71$.

Дальше визуализирую на графике сравнение метрик всех моделей на обучение, где лучше показывает себя RandomForest, остальные модели тоже показывают нормальную производительность.

На следующем графике визуализирую сравнение метрик всех моделей на тестовой выборке где лучше показала себя модель логистическая регрессия, но остальные не сильно отстают.

На следующем графике визуализирую сравнения моделей по ROC-AUC, где все три модели лучше случайного угадывания, потому что их ROC-кривые проходят выше диагонали случайного классификатора. 0.83 у логистической регрессии — это уже нормальный рабочий уровень, но не очень высокий, мало данных, при больших данных можно улучшить.

На следующем графике визуализирую сравнения моделей показатель переобучения $f1$, где у всех трех моделей есть переобучение по $f1$, у логистической регрессии оно минимально, хуже всего у RandomForest.

На следующем графике визуализирую сравнения моделей показатель переобучения ассигасу, где у всех трех моделей есть переобучение по ассигасу, у логистической регрессии оно минимально, хуже всего у RandomForest.

Базовая модель логистическая регрессия, показала себя лучше чем ансамбли, у логистической регрессии меньше всего переобучение и показывает лучшую производительность на тесте.

Рекомендации по продолжению исследования: Провести аналогичные исследования с большим количеством примеров. Можно изменить соотношение train/test.

Классификация: превышает ли значение SI медианное значение выборки

В классификации используется данные которые разбиты на train/test в пропорции 80/20. Сам датасет уже подготовлен в EDA и сохранен в csv. Этот датасет состоит примерно из 1001 объект и 168 столбцов, из которых целевой переменной - median_threshold, а также из признаков исключаются IC50, mM, CC50, mM и SI перед обучением моделей.

Визуализирую с помощью histplot распределение классов, видно что распределение по признаку median_threshold сбалансировано, класс 1 - значит превышает значение IC50 медианное значение выборки, класс 0 - не превышает.

За базовую модель беру логистическую регрессию. Использую StandardScaler. У модели есть переобучение, так как на тесте производительность заметно снизилась $f1 = 0.61$, качество модели нельзя назвать высоким. Модель одинаково находит все классы и одинаково ошибается так как precision и recall равны 0.61.

Еще для сравнения взял 2 модели ансамблей RandomForest и XGBoost.

Для XGBoost использовал RandomizedSearchCV, так как GridSearchCV подбирал очень долго, кросс-валидация = 3, лучшие параметры: доля объектов для каждого дерева = 0.8, число деревьев = 100, максимальная глубина дерева = 15, скорость обучения = 0.01, коэффициент L2-регуляризации = 1, доля признаков для каждого дерева = 0.7, коэффициент L1-регуляризации = 0.1. для XGBoost есть немалое переобучение, модель лучше находит класс который не превышает медианное значение $\text{recall} = 0.71$, чем класс который превышает, так как recall выше, хуже распознает класс который больше чем медианное значение $\text{recall} = 0.54$, часть объектов ошибочно относятся к классу который превышает медианное значение. Ошибается модель больше для класса который не

превышает медианное значение precision = 0.61, против класса который больше чем медианное значение precision = 0.65. Производительность модели на тесте низкая f1 = 0.62.

Для RandomForest использовал RandomizedSearchCV, кросс-валидация = 3, лучшие параметры: число деревьев = 300, минимальное число объектов, необходимое для разбиения узла = 2, минимальное число объектов в листе = 1, число признаков при разбиении = sqrt, максимальная глубина дерева = 10. Для RandomForest есть немало переобучение, модель лучше находит класс который не превышает медианное значение recall = 0.72, чем класс который превышает, так как recall выше, хуже распознает класс который больше чем медианное значение recall = 0.59, часть объектов ошибочно относится к классу который превышает медианное значение. Ошибается модель больше для класса который не превышает медианное значение precision = 0.64, против класса который больше чем медианное значение precision = 0.68. Производительность модели на тесте низкая f1 = 0.66.

Дальше визуализирую на графике сравнение метрик всех моделей на обучение, где лучше показывает себя показывают ансамбли.

На следующем графике визуализирую сравнение метрик всех моделей на тестовой выборке где лучше всех тесте показывает себя модель RandomForest лучше f1, accuracy, precision. Но логистическая регрессия показала самые ровные метрики.

На следующем графике визуализирую сравнения моделей по ROC-AUC, где лучше всего показала модель XGBoost = 0.69 и RandomForest = 0.68, а логистическая регрессия = 0.66.

На следующем графике визуализирую сравнения моделей показатель переобучения f1, где у всех трех моделей есть переобучение по f1, у логистической регрессии оно минимально, хуже всего у XGBoost.

На следующем графике визуализирую сравнения моделей показатель переобучения accuracy, где у всех трех моделей есть переобучение по f1, у логистической регрессии оно минимально, хуже всего у XGBoost.

Лучшей моделью оказалась базовая логистическая регрессия так как у нее не намного хуже метрики, но заметно меньше переобучение, но все модели показали низкую производительность, для улучшения производительности нужно больше данных.

Рекомендации по продолжению исследования: Провести аналогичные исследования с большим количеством примеров.

Классификация: превышает ли значение SI значение 8

В классификации используются данные которые разбиты на train/test в пропорции 90/10. Сам датасет уже подготовлен в EDA и сохранен в csv. Этот датасет состоит примерно из 1001 объект и 168 столбцов, из которых целевой переменной - threshold, а также из признаков исключаются IC50, mM, CC50, mM и SI перед обучением моделей.

Визуализирую с помощью histplot распределение классов, видно что распределение классов по признаку median_threshold имеет дисбаланс но не очень сильный, для сравнения в некоторые модели добавлю class_weight, класс 1 - значит превышает значение IC50 медианное значение выборки, класс 0 - не превышает.

Так как классы не сбалансированы дополнительно сделаю для моделей логистическая регрессия и RandomForest, балансировку класса (class_weight).

За базовую модель беру логистическую регрессию. Использую StandardScaler. У модели есть переобучение, модель значительно лучше находит класс который не превышает медианное значение recall = 0.78, чем который не превышает, так как recall выше, хуже распознает класс который больше чем медианное значение recall = 0.44, часть объектов ошибочно относится к классу который превышает медианное значение. Ошибается модель больше для класса который не превышает медианное значение precision = 0.72, против класса который больше чем медианное значение precision = 0.53.

Производительность модели низкая f1 = 0.62.

Логистическая регрессия с балансировкой класса на тестовой выборке стало хуже, по сравнению со стандартной моделью, есть большое переобучение, модель одинаково предсказывает оба класса precision и recall равным 0.58. Ошибается модель больше для класса который превышает медианное значение precision = 0.72, против класса который

меньше чем медианное значение precision = 0.44. Производительность модели низкая $f = 0.57$.

Еще для сравнения взял 2 модели ансамблей RandomForest и XGBoost.

Для XGBoost использовал RandomizedSearchCV, так как GridSearchCV подбирал очень долго, кросс-валидация = 3, лучшие параметры: доля объектов для каждого дерева = 1, число деревьев = 200, максимальная глубина дерева = 3, скорость обучения = 0.1, коэффициент L2-регуляризации = 10, доля признаков для каждого дерева = 0.7, коэффициент L1-регуляризации = 1. для XGBoost есть переобучение, модель значительно лучше находит класс который не превышает медианное значение recall = 0.85, чем класс который превышает, так как recall выше, хуже распознает класс который больше чем медианное значение recall = 0.50, часть объектов ошибочно относятся к классу который превышает медианное значение. Ошибается модель больше для класса который превышает медианное значение precision = 0.64, против класса который меньше чем медианное значение precision = 0.75. Производительность модели на тесте нормальная $f1 = 0.68$.

Для RandomForest с class_weight = balanced, использовал RandomizedSearchCV, кросс-валидация = 3, лучшие параметры: число деревьев = 100, минимальное число объектов, необходимое для разбиения узла = 2, минимальное число объектов в листе = 4, число признаков при разбиении = log2, максимальная глубина дерева = 7.

Для RandomForest с class_weight есть переобучение, модель лучше находит класс который не превышает медианное значение recall = 0.77, чем класс который превышает, так как recall выше, хуже распознает класс который больше чем медианное значение recall = 0.64, часть объектов ошибочно относятся к классу который превышает медианное значение. Ошибается модель больше для класса который превышает медианное значение precision = 0.61, против класса который больше чем медианное значение precision = 0.79. Производительность модели на тесте нормальная $f1 = 0.7$.

Для RandomForest использовал RandomizedSearchCV, кросс-валидация = 3, лучшие параметры: число деревьев = 100, минимальное число объектов, необходимое для разбиения узла = 2, минимальное число объектов в листе = 4, число признаков при разбиении = log2, максимальная глубина дерева = 7. для RandomForestClassifier class_weight не дал положительного эффекта, у стандартной модели есть переобучение, модель лучше находит класс который не превышает медианное значение recall = 0.77, чем класс который превышает, так как recall выше, хуже распознает класс который больше чем медианное значение recall = 0.64, часть объектов ошибочно относятся к классу который превышает медианное значение. Ошибается модель больше для класса который превышает медианное значение precision = 0.61, против класса который больше чем медианное значение precision = 0.79. Производительность модели на тесте нормальная $f1 = 0.7$.

Дальше визуализирую на графике сравнение метрик всех моделей на обучение, где лучше всех на обучении производительность у модели XGBoost, class_weight для логистической регрессии и для RandomForest, не дало положительного эффекта.

На следующем графике визуализирую сравнение метрик всех моделей на тестовой выборке где на тестовой выборке лучше показали ансамбли, class_weight для логистической регрессии и для RandomForest, не дало положительного эффекта.

На следующем графике визуализирую сравнения моделей по ROC-AUC, где по ROC-AUC лучшей моделью выглядит XGBoost, а оба варианта RandomForest идут почти сразу за ним, Logistic Regression заметно слабее, class_weight для логистической регрессии и для RandomForest, не дало положительного эффекта.

На следующем графике так как есть дисбаланс классов визуализирую сравнения моделей по PR-AUC, где так как есть дисбаланс класса добавил PR-AUC, лучшей моделью выглядит RandomForest, а XGBoost идет за ним, Logistic Regression заметно слабее, class_weight для логистической регрессии и для RandomForest, не дало положительного эффекта.

На следующем графике визуализирую сравнения моделей показатель переобучения $f1$, где у всех моделей есть переобучение, больше всего XGBoost, меньше всего у RandomForest, class_weight для логистической регрессии помог снизить переобучение.

На следующем графике визуализирую сравнения моделей показатель переобучения ассигасу, где у всех моделей есть переобучение, больше всего XGBoost и логистической регрессии с `class_weight`, `class_weight` увеличил переобучение для логистической регрессии, а для RandomForest нет.

Так как есть дисбаланс классов, для сравнения был добавлен `class_weight` для базовой модели логистической регрессии и RandomForest, `class_weight` не дало положительного эффекта, для логистической регрессии сделал даже хуже, больше переобучение и хуже метрики. Я думаю что `class_weight` не дал эффекта, потому что нет сильного дисбаланса класса. Лучшая модель это RandomForest, самое маленькое переобучение и лучше метрики.

Рекомендации по продолжению исследования: Провести аналогичные исследования с большим количеством примеров.